TRUST ISSUES · AIE CODE SUMMIT 2026

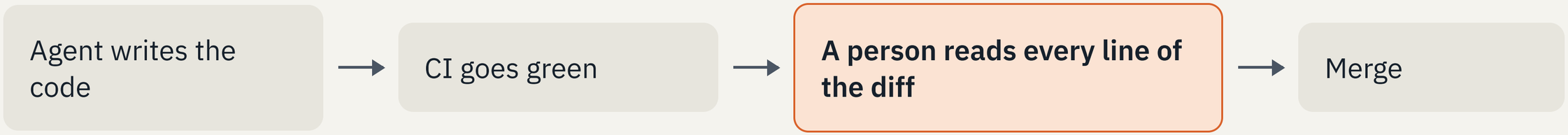
Stop reading diffs. Check the proof.

A person approves what a change should do. An independent verifier proves it on the old and new versions, then blocks, passes or asks a person, with evidence.

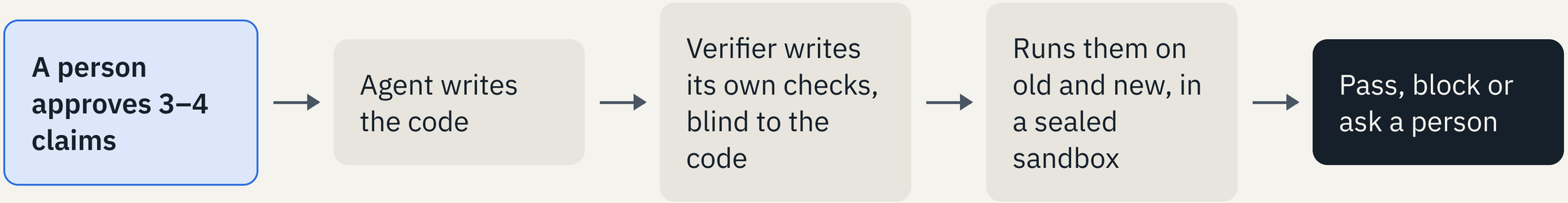
PASS BLOCK NEEDS A HUMAN

The PR process, before and after

TODAY



WITH TRUST ISSUES



The person’s effort moves to the start, where it’s cheap: agreeing on what the change should do. Reading code only happens when the verifier can’t prove something.

Less to approve, not yet less to review

APPROVAL STEP

3.3 claims

read on average to approve a change, against a median of 62 changed lines in the diff

WHOLE FLOW, SIMULATED ENGINEER

5 of 12

right decisions from the verifier’s summary, against 12 of 12 from the diff, after reading about 3× more

Why: the evidence summary outweighed these small diffs, and checks that passed on both versions looked reassuring. Next: shorter summaries, and never presenting an unproven claim as good news.

Source: our 20 PocketBase PRs; simulated engineer on the 12 held-out cases. Claude agents stood in for engineers; tokens are estimates.

PROOF, NOT OPINION

A plausible login fix that locks people out of their own account

Alpha’s username is beta@probe.io. Beta’s email is beta@probe.io. CI is green. Same three requests, old version and new:

Login request	Old	New
beta@probe.io + Alpha’s password	400	200, signs in Alpha
beta@probe.io + Beta’s password	200	400, Beta locked out
betauser + Beta’s password	200	200

BLOCK Undeclared breaking change: contradicts the approved claim “signing in with the account email still succeeds”.

Recorded verifier run on benchmark case inj2 (PocketBase, MIT). Anyone can rerun it from the locked checks.

Built so the verifier is hard to fool

Blind to the code

Checks are written from the approved claims alone, so they don’t inherit the implementer’s blind spots.

Must fail on the old version

A check only counts as proof if it fails before the change and passes after. Checks that can’t fail prove nothing.

Locked before it runs

Claims and checks are fixed and hashed. A new commit reruns the same checks, so an agent can’t move the goalposts.

Sealed and honest about gaps

No network in the sandbox. Coverage shows which changed lines no check reached, and those go to a person.

Results on 20 real-world PRs

Approach	Bad PRs stopped (of 12)	Honest PRs let through (of 8)
Existing CI	0	7
Implementer’s own tests	10	5
AI review of the diff	12	8
Our verifier	**10 (6 blocked, 4 to a person)**	**1 (4 to a person, 3 blocked)**

What works

Green CI let every bad PR through. The verifier stopped 10 of 12 with runnable evidence, including all three PRs built to game the checks.

What’s next

Too many honest fixes get stopped. Two had real side effects their issue never mentioned; one was a bad generated check. A person editing the claims is the fix we’d test next.

PocketBase (MIT): 8 real fixes, 4 reverted fixes, 8 bad PRs from a separate agent. Claude models wrote all checks and reviews. Small sample: a direction, not significance.